Lista 2 - Estrutura de Decisão Simples - if

Site: Ambiente Virtual de Aprendizagem - UFR
Course: ALGORITMOS E PROGRAMAÇÃO
Book: Lista 2 - Estrutura de Decisão Simples - if
Printed by: Nabor do Nascimento da Silva
Date: Sunday, 5 April 2026, 4:50 PM

Table of contents

1. Mostre os resultados dos testes lógicos
1.1. Tabela AND
1.2. Tabela OR

2. Estruturas simples e composta

3. Verificar se n = 0.
3.1. Estrutura simples
3.2. Estrutura composta

4. Verificar sinal de um numero
4.1. Estrutura simples
4.2. Estrutura composta

5. Verificar se um número é par ou impar
5.1. Estrutura simples
5.2. Estrutura composta

6. Verificar se denominador = 0
6.1. Estrutura simples
6.2. Estrutura composta

7. Verificar se delta = 0
7.1. Estrutura simples
7.2. Estrutura composta

8. a e b iguais ou diferentes
8.1. Estrutura simples
8.2. Estrutura composta

9. Estruturas encadeadas

10. positivo, negativo ou nulo
10.1. Solução

11. iguais, diferentes, maior, menor
11.1. Solução

12. Lados de um triângulo
12.1. Solução

13. Variáveis booleanas (lógicas)

14. Par ou impar
14.1. Solução

15. Condições de pagamento
15.1. Solução

16. Credenciais corretas
16.1. Solução

1. Mostre os resultados dos testes lógicos

Apresente os resultados para os exercícios a seguir.

1.1. Tabela AND

Execute os testes em Python:

Utilize o comando print.

Exemplo:

- print ((1+1 = 2) and (1>=0))

- (1+1 = 2) E (1>=0)
- (1+1 = 2) E (1>=3)
- (1+1 = 3) E (1>=0)
- (1+1 = 3) E (1>=3)

1.2. Tabela OR

Execute os testes em Python:

Utilize o comando print.

Exemplo:

- print ((1+1 = 2) OR (1>=0))

- (1+1 = 2) OU (1>=0)
- (1+1 = 2) OU (1>=3)
- (1+1 = 3) OU (1>=0)
- (1+1 = 3) OU (1>=3)

2. Estruturas simples e composta

Apresente os algoritmos para os exercícios a seguir, nas versões simples e composta.

3. Verificar se n = 0.

Elabore um algoritmo que:

- leia um número inteiro;
- verifique se o número digitado é igual a zero;
- informe ao usuário se o número é igual a zero ou não é igual a zero. Mostre o número lido na resposta ao usuário.

Caso 1

Exemplo de entrada esperada:

- Digite um número inteiro: 2

Exemplo de saída esperada:

- 2 não é igual a zero.

Caso 2

Exemplo de entrada esperada:

- Digite um número inteiro: 0

Exemplo de saída esperada:

- 0 é igual a zero.

3.1. Estrutura simples

```
#entrada de dados
n = int (input("digite um número inteiro "))
```

```
#processo
if (n==0):
    print (" é igual a zero")
if (n!=0):
    print (n, " não é igual a zero")
```

3.2. Estrutura composta

```
#entrada de dados
n = int (input("digite um número inteiro "))
```

```
#processo
if (n==0):
    print (" é igual a zero")
else
    print (n, " não é igual a zero")
```

4. Verificar sinal de um numero

Elabore um algoritmo que:

- leia um número inteiro diferente de zero
- verifique e informe ao usuário
  - se o número é maior que zero e, nesse caso, informar que o número é positivo.
  - se o número é menor que zero e, nesse caso, informar que o número é negativo.

Caso 1

Exemplo de entrada esperada:

- Digite um número inteiro: 2

Exemplo de saída esperada:

- 2 é positivo.

Caso 2

Exemplo de entrada esperada:

- Digite um número inteiro: -2

Exemplo de saída esperada:

- 2 é negativo.

4.1. Estrutura simples

```
#entrada de dados
n = int (input("digite um número inteiro diferente de zero"))

#processo
if (n>0):
   print (n, "é positivo")
if (n<0):
   print (n, "é negativo")
```

4.2. Estrutura composta

```
#entrada de dados
n = int (input("digite um número inteiro diferente de zero"))

#processo
if (n>0):
   print (n, "é positivo")
else:
   print (n, "é negativo")
```

5. Verificar se um número é par ou impar

Elabore um algoritmo que:

- leia um número inteiro;
- verifique se o número digitado é par ou impar;
- informe ao usuário se o número digitado é par ou impar. Mostre o número lido na resposta ao usuário.

Caso 1

Exemplo de entrada esperada:

- Digite um número inteiro: 2

Exemplo de saída esperada:

- 2 é par.

Caso 2

Exemplo de entrada esperada:

- Digite um número inteiro: 3

Exemplo de saída esperada:

- 3 é impar.

5.1. Estrutura simples

```
#entrada de dados
n = int (input("digite um número inteiro "))

#processo
if (n % 2 == 0):
    print (n, " é par")
if (n % 2 != 0):
    print (n, "  é impar")
```

5.2. Estrutura composta

```
#entrada de dados
n = int (input("digite um número inteiro "))

#processo
if (n % 2 == 0):
    print (n, " é par")
else:
    print (n, "  é impar")
```

6. Verificar se denominador = 0

Elabore um algoritmo que:

- leia um número real que será o numerador;
- leia um número real que será o denominador
- verifique se o denominador é zero.
- se o denominador não for zero, apresentar o resultado da divisão ao usuário.
- se o denominador for zero, informar ao usuário que não existe divisão por zero.

Caso 1

Exemplo de entradas esperadas:

- Digite numerador de uma divisão: 4
- Digite denominador de uma divisão: 0

Exemplo de saídas esperadas:

- Não existe divisão por zero.

Caso 2

Exemplo de entradas esperadas:

- Digite numerador de uma divisão: 4
- Digite denominador de uma divisão: 2

Exemplo de saídas esperadas:

- O resutlado da divisão é 2

6.1. Estrutura simples

```
#entrada de dados
numerador = int (input("digite o numerador "))
denominador = int (input("digite o denominador "))
```

```
#processo
if (denominador != 0):
    resultado = numerador / denominador
    print ("resultado da divisao = ",resultado)
if (denominador == 0):
    print ("não existe divisão por zero")
```

6.2. Estrutura composta

```
#entrada de dados
numerador = int (input("digite o numerador "))
denominador = int (input("digite o denominador "))
```

```
#processo
if (denominador != 0):
    resultado = numerador / denominador
    print ("resultado da divisao = ",resultado)
else:
    print ("não existe divisão por zero")
```

7. Verificar se delta = 0

Elabore um algoritmo que leia os coeficientes de uma equação de segundo grau completa, ou seja, do tipo $ax^2 + bx + c = 0$.

Utilizando a fórmula de Bhaskara:

1) leia os coeficientes a, b e c como entrada

2) verifique se delta é negativo

- se delta for negativo, informar que não existem raízes reais;
- se delta não for negativo, mostrar x1 e x2.

A fórmula de Bhaskara é dada por: $x = (-b \pm \sqrt{\Delta}) / 2a$, onde Δ (delta) = b**2 - 4*a*c

Caso 1:

Exemplos de entradas esperadas:

- Digite o coeficiente a: 2
- Digite o coeficiente b: -5
- Digite o coeficiente c: 2

Exemplo de saída esperada:

- Delta = 9
- Raízes da equação: 2.0 e 0.5

Caso 2:

Exemplos de entradas esperadas:

- Digite o coeficiente a: 4
- Digite o coeficiente b: -5
- Digite o coeficiente c: 2

Exemplo de saída esperada:

- Delta = -7
- Não existem raízes reais.

7.1. Estrutura simples

```
#entrada
a = int (input("digite o valor de a "))
b = int (input("digite o valor de b "))
c = int (input("digite o valor de c "))

#calculo de delta
delta = b**2 - (4*a*c)
print ("delta = ",delta)
if (delta <0):
   print ("não existem raizes reais")
if (delta >=0):
   x1 = (-b - delta**(1/2)) / (2*a)
   x2 = (-b + delta**(1/2)) / (2*a)
   print ("x1 = ", x1)
   print ("x2 = ", x2)
```

7.2. Estrutura composta

```
#entrada
a = int (input("digite o valor de a "))
b = int (input("digite o valor de b "))
c = int (input("digite o valor de c "))

#calculo de delta
delta = b**2 - (4*a*c)
print ("delta = ",delta)
if (delta <0):
   print ("não existem raizes reais")
else:
   x1 = (-b - delta**(1/2)) / (2*a)
   x2 = (-b + delta**(1/2)) / (2*a)
   print ("x1 = ", x1)
   print ("x2 = ", x2)
```

8. a e b iguais ou diferentes

Elabore um algoritmo que:

- leia dois números inteiros;
- verifique se os números são iguais ou diferentes;
- informe ao usuário se os números são iguais ou diferentes. Mostre os números lidos.

Caso 1:

Exemplos de entradas esperadas:

- Digite n1: 1
- Digite n2: 1

Exemplo de saída esperada:

- n1 igual a n2

Caso 2:

Exemplos de entradas esperadas:

- Digite n1 - inteiro: 1
- Digite n2 - inteiro: 2

Exemplo de saída esperada:

- n1 diferente de n2

8.1. Estrutura simples

```
#entrada de dados
n1 = int (input("digite n1 - inteiro"))
n2 = int (input("digite n2 - inteiro"))
```

```
#processo
if (n1 == n2):
    print ("n1 igual a n2")
if (n1 != n2):
    print ("n1 diferente de n2")
```

8.2. Estrutura composta

```
#entrada de dados
n1 = int (input("digite n1 - inteiro"))
n2 = int (input("digite n2 - inteiro"))
```

```
#processo
if (n1 == n2):
    print ("n1 igual a n2")
else:
    print ("n1 diferente de n2")
```

9. Estruturas encadeadas

Elabore algoritmos para as estruturas de decisão a seguir.

Utilize obrigatoriamente a estrutura composta encadeada.

10. positivo, negativo ou nulo

Elabore um algoritmo que verifique se um número real é negativo, positivo ou nulo.

Caso 1:

Entrada esperada:

- Digite um número: -1

Saída esperada:

- O número digitado é negativo.

Caso 2:

Entrada esperada:

- Digite um número: 1

Saída esperada:

- O número digitado é positivo.

Caso 3:

Entrada esperada:

- Digite um número: 0

Saída esperada:

- O número digitado é nulo.

10.1. Solução

```
#entrada
n = float (input("digite um número"))

#processo
if (n <0):
   print (n, " é negativo")
else:
   if (n>0):
      print (n, " é positivo")
   else:
      print (n, " é nulo")
```

11. iguais, diferentes, maior, menor

Elabore um algoritmo que verifique se 2 números inteiros são iguais ou diferentes. Caso sejam diferentes, informar qual é o maior e qual é o menor.

Caso 1:

Entrada esperada:

- Digite n1 - inteiro: 1
- Digite n2 - inteiro: 1

Saída esperada:

- n1 é igual a n2

Caso 2:

Entrada esperada:

- Digite n1 - inteiro: 1
- Digite n2 - inteiro: 2

Saída esperada:

- n1 é diferente de n2
- 1 é o menor
- 2 é o maior

Caso 3:

Entrada esperada:

- Digite n1 - inteiro: 2
- Digite n2 - inteiro: 1

Saída esperada:

- n1 é diferente de n2
- 2 é o maior
- 1 é o menor

11.1. Solução

```
#entrada
n1= int (input("Digite n1 - inteiro:"))
n2= int (input("Digite n2 - inteiro:"))

#processo
if (n1 == n2):
   print ("n1 é igual a n2")
else:
   if (n1>n2):
      print (n1, " é o maior")
      print (n2, " é o menor")
   else:
      print (n2, " é o maior")
      print (n1, " é o menor")
```

12. Lados de um triângulo

Para verificar se três lados a, b e c podem formar um triângulo, é necessário que satisfaçam a condição do triângulo, que é a seguinte:

- a+b>c, a+c>b, b+c>a

Ou seja, a soma de quaisquer dois lados deve ser maior que o terceiro lado.

Caso contrário,

- a, b e c não podem formar um triângulo.

Elabore um algoritmo que leia 3 valores reais e informe se eles formam um triângulo.

Caso 1: Triângulo válido

Exemplo de entradas esperadas:

- Digite o primeiro valor: 3
- Digite o segundo valor: 4
- Digite o terceiro valor: 5

Exemplo de saída esperada:

- Os valores 3.0, 4.0 e 5.0 formam um triângulo.

Caso 2: Triângulo inválido

Exemplo de entradas esperadas:

- Digite o primeiro valor: 2
- Digite o segundo valor: -3
- Digite o terceiro valor: 4

Exemplo de saída esperada:

- Os valores 2.0, -3.0 e 4.0 não formam um triângulo.

12.1. Solução

```
#entrada de dados
a = float (input ("digite o valor a"))
b = float (input ("digite o valor b"))
c = float (input ("digite o valor c"))
```

```
#processo
if (a+b>c and a+c>b and b+c>a):
    print ("Os valores ",a," ", b," e ",c, " formam um triângulo")
else:
    print ("Os valores ",a," ", b," e ",c, " não formam um triângulo")
```

13. Variáveis booleanas (lógicas)

Algoritmos de Decisões com Variáveis Booleanas envolvem problemas que precisam utilizar variáveis booleanas para armazenar o resultado de uma decisão e utilizá-las em outras partes do programa.

- Variáveis booleanas podem ser usadas para validar entradas ou estados em um programa.

Lembrando que um teste pode retornar true ou false, podemos ter:

- idade = 18
- print (idade>=18)

vai imprimir true.

Exemplo:

Elabore um algoritmo para receber a idade de uma pessoa e informar se é maior de idade ou não. Utilize uma variável booleana chamada idadeValida.

```
idade = int (input  ("digite sua idade"))
idadeValida = (idade >= 18)
if idadeValida:
    print("Você é maior de idade.")
else:
    print("Você é menor de idade.")
```

Elabore as soluções para os problemas desta categoria, apresentados a seguir.

14. Par ou impar

Escreva um algoritmo que receba um número inteiro como entrada e determine se o número é par ou ímpar.

Utilize uma variável booleana chamada par para armazenar o resultado da verificação.

Ao final, o algoritmo deve imprimir uma mensagem indicando se o número é par ou ímpar.

Exemplos de entradas e saídas válidas:

Caso 1: número par

Entradas:

- Digite um número: 2

Saida:

- 2 é par.

Caso 2: número impar

Entradas:

- Digite um número: 3

Saida:

- 3 é impar.

14.1. Solução

```
#entrada
n = int (input("digite um número"))
```

```
#processo
par = (n%2 == 0)
if (par):
    print (n," é par")
else:
    print (n," é impar")
```

15. Condições de pagamento

Desenvolva um algoritmo para uma loja online que verifica se um cliente é elegível para receber um desconto especial de acordo com as seguintes regras:

- O cliente deve ter feito mais de 5 compras.
- O valor total das compras do cliente deve ser superior a R$ 500,00.

Para isto, o algoritmo deve utilizar 2 variáveis booleanas:

- maisDe5Compras
- valorTotalAlto

Ao final, o algoritmo deve imprimir se o cliente é elegível para o desconto especial.

Exemplos de entradas e saídas válidas:

Caso 1: criente habilitado a receber desconto

Entradas:

- Digite o número de compras: 5
- Digite o valor total das compras: 500.00

Saida:

- Cliente pode receber desconto.

Caso 2: criente não habilitado a receber desconto (quantidade de compras inválido)

Entradas:

- Digite o número de compras: 4
- Digite o valor total das compras: 500.00

Saida:

- Cliente não pode receber desconto.

Caso 3: criente não habilitado a receber desconto (valor total de compras inválido)

Entradas:

- Digite o número de compras: 5
- Digite o valor total das compras: 400.00

Saida:

- Cliente não pode receber desconto.

Caso 4: criente não habilitado a receber desconto (quantidade de compras e valor total de compras inválidos)

Entradas:

- Digite o número de compras: 4
- Digite o valor total das compras: 400.00

Saida:

- Cliente não pode receber desconto.

15.1. Solução

```
#entrada
maisDe5Compras = int (input("digite a qtd de compras"))
valorTotalAlto = int (input("digite o valor total das compras"))

#processo
if (maisDe5Compras >= 5) and (valorTotalAlto >=500.00):
    print ("cliente pode receber desconto")
else:
    print ("cliente não pode receber desconto")
```

16. Credenciais corretas

Implemente um algoritmo para um sistema de login que verifica se um usuário tem acesso administrativo baseado nas seguintes condições:

- O usuário deve ter inserido corretamente sua senha. A senha = 123
- O usuário deve ser maior de 18 anos de idade.

Utilize variáveis booleanas:

- senha e
- idade

para armazenar o resultado de cada verificação.

Ao final, o algoritmo deve imprimir se o usuário possui acesso administrativo.

Exemplos de entradas e saídas válidas:

Caso 1: senha e idade válidos

Entradas:

- Digite sua senha: 123
- Digite sua idade: 18

Saida:

- Usuário credenciado.

Caso 2: senha correta e idade inválida

Entradas:

- Digite sua senha: 123
- Digite sua idade: 12

Saida:

- Usuário não credenciado.

Caso 3: senha incorreta e idade válida

Entradas:

- Digite sua senha: 1234
- Digite sua idade: 18

Saida:

- Usuário não credenciado.

16.1. Solução

```
#entrada
senha = int (input("digite a senha"))
idade = int (input("digite a idade"))
```

```
#processo
if (senha == 123) and (idade >=18):
    print ("usuário credenciado")
else:
    print ("usuário não credenciado")
```